

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Курсовая работа по курсу

«Операционные системы»

Группа: М8О-209БВ-24

Студент: Буланов М.С.

Преподаватель: Миронов Е.С.

Оценка: _____

Дата: 16.12.25

Москва, 2025

Постановка задачи

Вариант 7.

Консоль-серверная игра. Необходимо написать консоль-серверную игру. Необходимо написать 2 программы: сервер и клиент. Сначала запускается сервер, а далее клиенты соединяются с сервером. Сервер координирует клиентов между собой. При запуске клиента игрок может выбрать одно из следующих действий (возможно больше, если предусмотрено вариантом):

Создать игру, введя ее имя

Присоединиться к одной из существующих игр по имени игры

«Быки и коровы» (угадывать необходимо слова). Общение между сервером и клиентом необходимо организовать при помощи pipe'ов. При создании каждой игры необходимо указывать количество игроков, которые будут участвовать. То есть угадывать могут несколько игроков. Если кто-то из игроков вышел из игры, то игра должна быть продолжена.

Общий метод и алгоритм решения

Программа реализует локальную многопользовательскую игру «Быки и коровы» в архитектуре **клиент–сервер** с использованием **именованных каналов FIFO (named pipes)**.

Сервер хранит список активных игр, состав игроков и скрытое слово каждой игры. Клиенты отправляют команды серверу через общий FIFO `server_fifo`, а ответы получают через индивидуальный FIFO клиента `client_<pid>_fifo`.

Для параллельного ввода/вывода на стороне клиента используется **второй поток**, который читает сообщения от сервера и выводит их в консоль, пока основной поток принимает пользовательский ввод.

Использованные системные вызовы

(Часть вызывается напрямую через POSIX API, часть — косвенно через `libc/stdio/threads`.)

IPC и работа с файлами/FIFO:

- `mkfifo()` — создание именованного канала FIFO (`server_fifo` и `client_<pid>_fifo`)
- `unlink()` — удаление FIFO при завершении процесса
- `open()` / (на уровне ядра обычно `openat()`) — открытие FIFO/файлов
- `read()` — чтение команд/сообщений из FIFO
- `write()` — запись команд/сообщений в FIFO
- `close()` — закрытие файловых дескрипторов

Ожидание событий и тайминги:

- `select()` — ожидание появления данных в `server_fifo` на стороне сервера
- `sleep()` / `usleep()` — небольшие задержки в клиенте (ожидание вывода списка/снижение нагрузки на CPU)

Идентификация процесса:

- `getpid()` — получение PID клиента для формирования имени `client_<pid>_fifo`

Сигналы и завершение:

- `signal()` / (на уровне ядра `rt_sigaction`) — обработка `SIGINT` на сервере
- `exit()` / (на уровне ядра `exit_group()`) — завершение процесса

Потоки (косвенно через `pthread`):

- `pthread_create()` — создание потока чтения в клиенте
(на уровне ядра обычно появляются `clone()` и синхронизация через `futex()`)

Алгоритм работы программы

• 1) Запуск сервера

1. Сервер удаляет старый FIFO `server_fifo` (если остался) и создаёт новый через `mkfifo("server_fifo", 0666)`.
2. Открывает `server_fifo` в неблокирующем режиме `open(..., O_RDONLY | O_NONBLOCK)`.
3. Переходит в цикл ожидания команд:
 - `select()` ждёт, когда в `server_fifo` появятся данные;
 - затем сервер делает `read()` и разбирает команду через `sscanf()`.

Дополнительно сервер ведёт лог в файл `server.log` (через `log_event()`).

• 2) Запуск клиента

1. Клиент получает свой PID: `pid = getpid()`.
 2. Создаёт личный FIFO: `client_<pid>_fifo`.
 3. Открывает `server_fifo` на запись (`O_WRONLY`).
 4. Открывает свой FIFO на чтение (`O_RDONLY | O_NONBLOCK`).
 5. Запускает поток `read_thread_func`, который постоянно читает `client_<pid>_fifo` и печатает сообщения от сервера.
-

• 3) Протокол команд (клиент → сервер)

Клиент отправляет строковые команды в `server_fifo`:

- Создание игры:
`CREATE <game_name> <max_players> <pid>`
- Запрос списка игр:
`LIST <pid>`
- Вход в игру:
`JOIN <game_name> <pid>`
- Попытка угадать слово:
`GUESS <game_name> <pid> <word>`

- Выход из игры:
`EXIT <game_name> <pid>`
-

• 4) Обработка команд на сервере

CREATE

1. Сервер проверяет:
 - не превышен лимит количества игр;
 - `max_players` находится в диапазоне (в текущей версии — 2..5);
 - имя игры уникально.
2. Создаёт новую игру:
 - выбирает секретное слово из `words.txt` (`random_word()`);
 - добавляет создателя как первого игрока;
 - устанавливает `current_turn = 0`.
3. Отправляет клиенту ответ “Game created successfully”.

LIST

- Сервер формирует строку со списком активных игр и количеством игроков `Players: current/max`.
- Отправляет список клиенту в его FIFO.

JOIN

1. Сервер проверяет:
 - существует ли игра;
 - не заполнена ли игра.
2. Добавляет игрока в список игроков игры.
3. Отправляет клиенту “Joined game successfully” и делает broadcast уведомления игрокам “Player X joined...”.

GUESS

1. Сервер проверяет:
 - существует ли игра;
 - есть ли игроки;
 - ходит ли сейчас этот игрок (`players[current_turn].pid == pid`);
 - совпадает ли длина слова с длиной секретного.
2. Считает “быков” и “коров” функцией `bulls_cows()`.
3. Делает broadcast всем игрокам игры с результатом вида:
`Player <pid> guessed "<word>": Bulls=N, Cows=M`
4. Если `Bulls == длина_слова`, сервер сообщает о победе:
`Player <pid> WINS the game!`
и пишет событие в лог.

EXIT

1. Сервер удаляет игрока из игры (`remove_player()`).
2. Делает broadcast “Player X exited...”.
3. При необходимости может сдвинуть `current_turn` так, чтобы индекс хода не выходил за пределы списка игроков.

• 5) Рассылка ответов сервер → клиент

- Сервер отвечает клиентам через их FIFO `client_<pid>_fifo`.
- Для отправки сервер делает: `open(O_WRONLY | O_NONBLOCK) → write() → close()`.

• 6) Логирование

Сервер пишет события в `server.log` в формате:

- текущая версия:
`[unix_timestamp] message`
Например:
`[1703100000] Game created`

(Если потребуется, формат можно изменить на ISO-8601, но в этих файлах используется Unix time.)

Обоснование выбранного подхода

1. **FIFO** — простой и наглядный POSIX-механизм IPC для локального взаимодействия процессов без сетевых сокетов.
2. **Раздельные FIFO** (один общий для команд + отдельный для каждого клиента) упрощают маршрутизацию ответов и позволяют обслуживать несколько клиентов.
3. **select() на сервере** позволяет эффективно ждать входные команды без постоянного опроса (без busy-wait).
4. **Поток чтения в клиенте** предотвращает блокировку ввода: серверные сообщения отображаются сразу, даже когда пользователь вводит слово.
5. **Мьютекс print_mutex** в клиенте нужен, чтобы вывод из потоков не перемешивался.

Код программы

Server.c

```
#define _POSIX_C_SOURCE 200809L
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/select.h>
#include <pthread.h>
#include <time.h>
#include <signal.h>
#include <errno.h>
```

```

static void sleep_ms(long ms) {
    if (ms <= 0) return;
    struct timespec ts;
    ts.tv_sec = ms / 1000;
    ts.tv_nsec = (ms % 1000) * 1000000L;
    while (nanosleep(&ts, &ts) == -1 && errno == EINTR) {
        // continue sleeping for remaining time
    }
}

#define SERVER_FIFO "server_fifo"
#define MAX_GAMES 10
#define MAX_PLAYERS 5
#define WORD_LEN 32

typedef struct {
    pid_t pid;
} Player;

typedef struct {
    char name[32];
    char secret[WORD_LEN];
    int max_players;
    int player_count;
    int current_turn;
    Player players[MAX_PLAYERS];
} Game;

static Game games[MAX_GAMES];
static int game_count = 0;
static pthread_mutex_t games_mutex = PTHREAD_MUTEX_INITIALIZER;

static int server_fd = -1;

// Main loop + background threads stop when this flag becomes 0.
static volatile sig_atomic_t running = 1;

// If shutdown requested by a signal, we store it here for logging in main().
static volatile sig_atomic_t shutdown_signal = 0;

static pthread_t reaper_thread;
static int reaper_started = 0;

static void log_event(const char *msg) {
    FILE *f = fopen("server.log", "a");
    if (!f) return;

    time_t now = time(NULL);
    struct tm tm_local;
    localtime_r(&now, &tm_local);

    char dt[32];
    if (strftime(dt, sizeof(dt), "%Y-%m-%dT%H:%M:%S", &tm_local) == 0) {
        // Fallback: should be very rare
    }
}

```

```

        fprintf(f, "[%ld] %s\n", (long)now, msg);
        fclose(f);
        return;
    }

    // %z produces +hhmm / -hhmm. Convert to +hh:mm for ISO-8601 readability.
    char tz_raw[8];
    tz_raw[0] = 0;
    (void)strftime(tz_raw, sizeof(tz_raw), "%z", &tm_local);

    char tz_iso[8];
    tz_iso[0] = 0;
    if (strlen(tz_raw) == 5) {
        tz_iso[0] = tz_raw[0];
        tz_iso[1] = tz_raw[1];
        tz_iso[2] = tz_raw[2];
        tz_iso[3] = ':';
        tz_iso[4] = tz_raw[3];
        tz_iso[5] = tz_raw[4];
        tz_iso[6] = 0;
    }

    fprintf(f, "[%s%s] %s\n", dt, tz_iso[0] ? tz_iso : "", msg);
    fclose(f);
}

```

```

// Best-effort cleanup for normal exit and most crashes.
// NOTE: cleanup cannot be guaranteed for SIGKILL (kill -9) or power loss.

```

```

static void server_cleanup(void) {
    if (server_fd >= 0) {
        close(server_fd);
        server_fd = -1;
    }
    unlink(SERVER_FIFO);
}

```

```

static void server_crash_handler(int sig) {
    // Async-signal-safe best effort: close fd then exit immediately.
    // We deliberately do NOT unlink the FIFO here (not async-signal-safe).
    // The next server start unlinks SERVER_FIFO before mkfifo().
    if (server_fd >= 0) {
        close(server_fd);
        server_fd = -1;
    }
    _exit(128 + sig);
}

```

```

static void server_graceful_handler(int sig) {
    shutdown_signal = sig;
    running = 0;
}

```

```

    // Wake select(): close is async-signal-safe.
    if (server_fd >= 0) {
        close(server_fd);
        server_fd = -1;
    }
}

static void install_server_handlers(void) {
    atexit(server_cleanup);

    // Graceful-ish exits (stop loops, then main joins threads & cleans up)
    signal(SIGINT, server_graceful_handler);
    signal(SIGTERM, server_graceful_handler);
    signal(SIGHUP, server_graceful_handler);
    signal(SIGQUIT, server_graceful_handler);

    // Common crash signals (best-effort cleanup; cannot guarantee)
    signal(SIGSEGV, server_crash_handler);
    signal(SIGABRT, server_crash_handler);
    signal(SIGFPE, server_crash_handler);
    signal(SIGILL, server_crash_handler);
#ifdef SIGBUS
    signal(SIGBUS, server_crash_handler);
#endif

    // Don't die if we write to a broken pipe/FIFO.
    signal(SIGPIPE, SIG_IGN);
}

static void remove_game_by_index(int idx) {
    if (idx < 0 || idx >= game_count) return;
    for (int i = idx; i < game_count - 1; i++) {
        games[i] = games[i + 1];
    }
    game_count--;
}

static void prune_empty_games_locked(void) {
    // Must be called with games_mutex held.
    for (int i = 0; i < game_count; ) {
        if (games[i].player_count <= 0) {
            char logbuf[128];
            snprintf(logbuf, sizeof(logbuf), "Game '%s' ended (no players)",
games[i].name);
            log_event(logbuf);
            remove_game_by_index(i);
            continue;
        }
        i++;
    }
}

static Game *find_game(const char *name) {

```



```

    for (int i = 0; i < game_count; i++) {
        if (!strcmp(games[i].name, name)) return &games[i];
    }
    return NULL;
}

static int player_index(Game *g, pid_t pid) {
    for (int i = 0; i < g->player_count; i++) {
        if (g->players[i].pid == pid) return i;
    }
    return -1;
}

static void remove_player(Game *g, pid_t pid) {
    int idx = player_index(g, pid);
    if (idx < 0) return;

    for (int j = idx; j < g->player_count - 1; j++) {
        g->players[j] = g->players[j + 1];
    }

    g->player_count--;
    if (g->player_count <= 0) {
        g->current_turn = 0;
        return;
    }

    // If the removed player was before current_turn, shift current_turn left.
    if (idx < g->current_turn) {
        g->current_turn--;
        if (g->current_turn < 0) g->current_turn = 0;
    }

    if (g->current_turn >= g->player_count) {
        g->current_turn = 0;
    }
}

static void random_word(char *word) {
    FILE *f = fopen("words.txt", "r");
    if (!f) {
        strcpy(word, "apple");
        return;
    }

    char words[100][WORD_LEN];
    int count = 0;
    while (count < 100 && fscanf(f, "%31s", words[count]) == 1) {
        count++;
    }
    fclose(f);

    if (count == 0) {
        strcpy(word, "apple");
    }
}

```

```

        return;
    }

    strcpy(word, words[rand() % count]);
}

// Returns 0 on success, -1 on failure.
static int send_to_client(pid_t pid, const char *msg) {
    char fifo[64];
    snprintf(fifo, sizeof(fifo), "client_%d_fifo", pid);

    int fd = open(fifo, O_WRONLY | O_NONBLOCK);
    if (fd >= 0) {
        (void)write(fd, msg, strlen(msg));
        close(fd);
        return 0;
    }

    // If open() fails with ENOENT/ENXIO, the client FIFO is missing or has no reader
    // (common when client crashed). Treat as dead.
    if (errno == ENOENT || errno == ENXIO) {
        (void)unlink(fifo); // best-effort cleanup of stale FIFO
    }

    char logbuf[160];
    snprintf(logbuf, sizeof(logbuf), "Failed to send to client %d (errno=%d)", pid,
errno);
    log_event(logbuf);
    return -1;
}

static void broadcast_to_game(Game *g, const char *msg) {
    // If a client disappeared (Ctrl+C, crash), its FIFO will be gone.
    // We remove such players to keep game state consistent.
    pid_t dead[MAX_PLAYERS];
    int dead_count = 0;

    for (int i = 0; i < g->player_count; i++) {
        if (send_to_client(g->players[i].pid, msg) < 0) {
            dead[dead_count++] = g->players[i].pid;
        }
    }

    for (int i = 0; i < dead_count; i++) {
        remove_player(g, dead[i]);
    }
}

static void bulls_cows(const char *secret, const char *guess, int *b, int *c) {
    *b = *c = 0;
    int len = (int)strlen(secret);

    for (int i = 0; i < len; i++) {
        if (guess[i] == secret[i]) {

```

```

        (*b)++;
    } else {
        for (int j = 0; j < len; j++) {
            if (j != i && guess[i] == secret[j]) {
                (*c)++;
                break;
            }
        }
    }
}
}
}
}

```

// Reaper: periodically removes disconnected clients and prunes empty games.
 // Runs in a separate thread and exits when `running` becomes 0.

```

static int client_fifo_has_reader(pid_t pid) {
    char fifo[64];
    snprintf(fifo, sizeof(fifo), "client_%d_fifo", pid);

    int fd = open(fifo, O_WRONLY | O_NONBLOCK);
    if (fd >= 0) {
        close(fd);
        return 1;
    }
}

```

```

// ENOENT: fifo doesn't exist, ENXIO: exists but no reader
if (errno == ENOENT || errno == ENXIO) return 0;

// On other errors, assume alive to avoid false kicks.
return 1;
}

```

```

static void *reaper_thread_func(void *arg) {
    (void)arg;

    while (running) {
        // Sleep up to ~2 seconds, but remain responsive to shutdown.
        for (int i = 0; i < 20 && running; i++) {
            sleep_ms(100);
        }
        if (!running) break;

        pthread_mutex_lock(&games_mutex);
        for (int gi = 0; gi < game_count; gi++) {
            Game *g = &games[gi];
            for (int pi = 0; pi < g->player_count; ) {
                pid_t pid = g->players[pi].pid;
                if (!client_fifo_has_reader(pid)) {
                    char logbuf[160];
                    snprintf(logbuf, sizeof(logbuf), "Player %d removed (disconnected)",
pid);

                    log_event(logbuf);
                    remove_player(g, pid);
                    continue; // keep same pi
                }
            }
        }
    }
}

```

```

        pi++;
    }
}
prune_empty_games_locked();
pthread_mutex_unlock(&games_mutex);
}

return NULL;
}

```

```

static void handle_command(const char *cmdline) {
    if (!cmdline) return;

    // Trim leading whitespace
    while (*cmdline == ' ' || *cmdline == '\t' || *cmdline == '\r') cmdline++;
    if (*cmdline == '\0') return;

    char buf[256];
    strncpy(buf, cmdline, sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = '\0';

    // Trim trailing CR (in case of \r\n)
    size_t blen = strlen(buf);
    if (blen > 0 && buf[blen - 1] == '\r') buf[blen - 1] = '\0';

    char game_name[32], word[WORD_LEN];
    pid_t pid;
    int max_players;

    pthread_mutex_lock(&games_mutex);

    if (sscanf(buf, "CREATE %31s %d %d", game_name, &max_players, &pid) == 3) {
        if (game_count >= MAX_GAMES) {
            send_to_client(pid, "Max games reached\n");
            pthread_mutex_unlock(&games_mutex);
            return;
        }
        if (max_players < 1 || max_players > MAX_PLAYERS) {
            send_to_client(pid, "Invalid max players (1-5)\n");
            pthread_mutex_unlock(&games_mutex);
            return;
        }
        if (find_game(game_name) != NULL) {
            send_to_client(pid, "Game name taken\n");
            pthread_mutex_unlock(&games_mutex);
            return;
        }

        Game *g = &games[game_count++];
        strcpy(g->name, game_name);
        random_word(g->secret);
        g->max_players = max_players;
        g->player_count = 1;
    }
}

```

```

g->current_turn = 0;
g->players[0].pid = pid;

send_to_client(pid, "Game created successfully\n");
log_event("Game created");
}
else if (sscanf(buf, "JOIN %31s %d", game_name, &pid) == 2) {
    Game *g = find_game(game_name);
    if (!g) {
        send_to_client(pid, "Game not found\n");
    } else if (player_index(g, pid) >= 0) {
        send_to_client(pid, "Already in this game\n");
    } else if (g->player_count >= g->max_players) {
        send_to_client(pid, "Game is full\n");
    } else {
        g->players[g->player_count++].pid = pid;
        send_to_client(pid, "Joined game successfully\n");

        char msg[96];
        snprintf(msg, sizeof(msg), "Player %d joined the game.\n", pid);
        broadcast_to_game(g, msg);
        log_event("Player joined");
    }
}
else if (sscanf(buf, "GUESS %31s %d %31s", game_name, &pid, word) == 3) {
    Game *g = find_game(game_name);
    if (!g) {
        send_to_client(pid, "Game not found\n");
    } else {
        int idx = player_index(g, pid);
        if (idx < 0) {
            send_to_client(pid, "You are not in this game\n");
        }
        else if (g->player_count <= 0) {
            send_to_client(pid, "Game is empty\n");
        }
        else if (g->players[g->current_turn].pid != pid) {
            send_to_client(pid, "Not your turn\n");
        }
        else if ((int)strlen(word) != (int)strlen(g->secret)) {
            send_to_client(pid, "Wrong word length\n");
        }
        else {
            int b, c;
            bulls_cows(g->secret, word, &b, &c);

            char msg[256];
            snprintf(msg, sizeof(msg), "Player %d guessed \"%s\": Bulls=%d,
Cows=%d\n", pid, word, b, c);
            broadcast_to_game(g, msg);

            if (b == (int)strlen(g->secret)) {
                snprintf(msg, sizeof(msg), "Player %d WINS the game!\n", pid);
                broadcast_to_game(g, msg);
            }
        }
    }
}

```

```

        broadcast_to_game(g, "Game ended.\n");
        log_event("Game won");

        int gi = (int)(g - games);
        remove_game_by_index(gi);
        prune_empty_games_locked();
        pthread_mutex_unlock(&games_mutex);
        return;
    }

    if (g->player_count > 0) {
        g->current_turn = (g->current_turn + 1) % g->player_count;
    } else {
        g->current_turn = 0;
    }
}

}

else if (sscanf(buf, "EXIT %31s %d", game_name, &pid) == 2) {
    Game *g = find_game(game_name);
    if (g) {
        if (player_index(g, pid) < 0) {
            send_to_client(pid, "You are not in this game\n");
        } else {
            char msg[96];
            snprintf(msg, sizeof(msg), "Player %d exited the game.\n", pid);
            remove_player(g, pid);
            broadcast_to_game(g, msg);
            log_event("Player exited");
        }
    }
    prune_empty_games_locked();
}

else if (sscanf(buf, "LIST %d", &pid) == 1) {
    if (game_count == 0) {
        send_to_client(pid, "No active games\n");
    } else {
        char msg[1024] = "Active games:\n";
        for (int i = 0; i < game_count; i++) {
            char line[128];
            snprintf(line, sizeof(line), "Game: %s Players: %d/%d\n",
                     games[i].name, games[i].player_count, games[i].max_players);
            strncat(msg, line, sizeof(msg) - strlen(msg) - 1);
        }
        send_to_client(pid, msg);
    }
}

prune_empty_games_locked();

pthread_mutex_unlock(&games_mutex);
}

int main(void) {

```

```

install_server_handlers();

// Seed RNG once (POSIX-style): random_word() should not reseed each call.
srand((unsigned)time(NULL) ^ (unsigned)getpid());

// If last run crashed and FIFO is still present
unlink(SERVER_FIFO);

if (mkfifo(SERVER_FIFO, 0666) < 0) {
    perror("mkfifo");
    return 1;
}

// Open as O_RDWR to avoid EOF/spin when there are no clients.
server_fd = open(SERVER_FIFO, O_RDWR | O_NONBLOCK);
if (server_fd < 0) {
    perror("open server fifo");
    unlink(SERVER_FIFO);
    return 1;
}

// Background cleanup so games end even if clients crash silently.
if (pthread_create(&reaper_thread, NULL, reaper_thread_func, NULL) == 0) {
    reaper_started = 1;
}

log_event("Server started");

fd_set rfd;

char rbuf[256];
char inbuf[2048];
size_t inlen = 0;

while (running) {
    if (server_fd < 0) break;

    FD_ZERO(&rfd);
    FD_SET(server_fd, &rfd);

    int ret = select(server_fd + 1, &rfd, NULL, NULL, NULL);
    if (ret < 0) {
        if (!running) break; // shutdown requested
        if (errno == EBADF) break; // fd closed in signal handler
        perror("select");
        break;
    }

    if (!FD_ISSET(server_fd, &rfd)) continue;

    int n = read(server_fd, rbuf, sizeof(rbuf));
    if (n <= 0) continue;

    // Append into line buffer

```

```

if (inlen + (size_t)n >= sizeof(inbuf)) {
    // Buffer overflow: drop accumulated data (protocol error / flood).
    inlen = 0;
}
memcpy(inbuf + inlen, rbuf, (size_t)n);
inlen += (size_t)n;

// Process complete lines (commands end with '\n')
for (;;) {
    void *p = memchr(inbuf, '\n', inlen);
    if (!p) break;

    size_t linelen = (size_t)((char *)p - inbuf);
    char line[512];

    if (linelen >= sizeof(line)) {
        linelen = sizeof(line) - 1;
    }
    memcpy(line, inbuf, linelen);
    line[linelen] = '\0';

    // Shift buffer past this line + '\n'
    size_t consumed = (size_t)(((char *)p - inbuf) + 1);
    size_t remaining = inlen - consumed;
    memmove(inbuf, inbuf + consumed, remaining);
    inlen = remaining;

    handle_command(line);
}
}

// Stop background thread and exit cleanly.
running = 0;
if (reaper_started) {
    pthread_join(reaper_thread, NULL);
}

// Log server shutdown (graceful stop).
if (shutdown_signal) {
    char msg[128];
    snprintf(msg, sizeof(msg), "Server stopped (signal %d)", (int)shutdown_signal);
    log_event(msg);
} else {
    log_event("Server stopped");
}

server_cleanup();
return 0;
}

```


client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>
#include <pthread.h>
#include <signal.h>
#include <errno.h>
#include <time.h>

#define SERVER_FIFO "server_fifo"
#define MAX_PLAYERS 5

static int server_fd = -1;
static int client_fd = -1;

static char game[32] = {0};
static pid_t pid;
static char client_fifo_path[64] = {0};

static volatile sig_atomic_t running = 1;
static volatile sig_atomic_t list_running = 0;
static volatile sig_atomic_t game_over = 0;

static pthread_t main_thread_id;

static pthread_mutex_t print_mutex = PTHREAD_MUTEX_INITIALIZER;

// Synchronization to wait for CREATE/JOIN result
static pthread_mutex_t resp_mutex = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t resp_cond = PTHREAD_COND_INITIALIZER;
static int waiting_create = 0;
static int waiting_join = 0;
static int last_result = 0; // 0=unknown, 1=success, -1=failure

// Prebuilt EXIT message used by SIGINT handler (async-signal-safe write()).
static char exit_msg[128] = {0};
static volatile sig_atomic_t exit_msg_len = 0;
static volatile sig_atomic_t have_game = 0;

static void safe_print(const char *msg) {
    pthread_mutex_lock(&print_mutex);
    fputs(msg, stdout);
    fflush(stdout);
    pthread_mutex_unlock(&print_mutex);
}

static void client_cleanup(void) {
    if (client_fd >= 0) {
        close(client_fd);
        client_fd = -1;
    }
}
```

```

    }
    if (server_fd >= 0) {
        close(server_fd);
        server_fd = -1;
    }
    if (client_fifo_path[0] != '\0') {
        unlink(client_fifo_path);
    }
}

static void on_signal(int sig) {
    (void)sig;
    running = 0;
    list_running = 0;

    // Best-effort: tell server we're leaving the current game.
    if (have_game && server_fd >= 0 && exit_msg_len > 0) {
        (void)write(server_fd, exit_msg, (size_t)exit_msg_len);
    }

    client_cleanup();
    _exit(128 + sig);
}

// Used only to interrupt blocking stdin reads (fgets/scanf) from another thread.
static void wake_stdin_handler(int sig) {
    (void)sig;
}

static void install_client_handlers(void) {
    atexit(client_cleanup);

    signal(SIGINT, on_signal);
    signal(SIGTERM, on_signal);
    signal(SIGHUP, on_signal);
    signal(SIGQUIT, on_signal);

    // Crash signals: best-effort cleanup (can't be guaranteed for all cases)
    signal(SIGSEGV, on_signal);
    signal(SIGABRT, on_signal);
    signal(SIGFPE, on_signal);
    signal(SIGILL, on_signal);
#ifdef SIGBUS
    signal(SIGBUS, on_signal);
#endif

    // Don't die if we write to a closed FIFO.
    signal(SIGPIPE, SIG_IGN);

    // Used by reader thread to wake the main thread if the game ends.
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = wake_stdin_handler;
    sigemptyset(&sa.sa_mask);

```

```

    sa.sa_flags = 0; // important: do NOT use SA_RESTART so fgets can be interrupted
    sigaction(SIGUSR1, &sa, NULL);
}

static void maybe_signal_waiters(const char *msg) {
    // Called from reader thread.
    pthread_mutex_lock(&resp_mutex);

    if (waiting_create) {
        if (strstr(msg, "Game created successfully") != NULL) {
            last_result = 1;
            waiting_create = 0;
            pthread_cond_signal(&resp_cond);
        } else if (strstr(msg, "Max games") || strstr(msg, "Invalid") || strstr(msg,
"taken")) {
            last_result = -1;
            waiting_create = 0;
            pthread_cond_signal(&resp_cond);
        }
    }

    if (waiting_join) {
        if (strstr(msg, "Joined game successfully") != NULL) {
            last_result = 1;
            waiting_join = 0;
            pthread_cond_signal(&resp_cond);
        } else if (strstr(msg, "Game is full") || strstr(msg, "Game not found") ||
strstr(msg, "Already in") ) {
            last_result = -1;
            waiting_join = 0;
            pthread_cond_signal(&resp_cond);
        }
    }

    pthread_mutex_unlock(&resp_mutex);
}

static void maybe_handle_game_over(const char *msg) {
    if (game_over) return;

    // Server broadcasts these when the game finishes.
    if (strstr(msg, "Game ended") != NULL || strstr(msg, "WINS the game") != NULL) {
        game_over = 1;
        have_game = 0;
        exit_msg_len = 0;
        list_running = 0;

        safe_print("\n[Client] Game finished. Exiting input mode...\n");

        // Wake the main thread if it's blocked on stdin.
        pthread_kill(main_thread_id, SIGUSR1);
    }
}

```

```

static void *read_thread_func(void *arg) {
    (void)arg;
    char buf[256];

    while (running) {
        int n = read(client_fd, buf, sizeof(buf) - 1);
        if (n > 0) {
            buf[n] = '\0';

            maybe_signal_waiters(buf);
            maybe_handle_game_over(buf);

            pthread_mutex_lock(&print_mutex);
            printf("\r%s\n> ", buf);
            fflush(stdout);
            pthread_mutex_unlock(&print_mutex);
        } else {
            usleep(100000);
        }
    }

    return NULL;
}

static void *list_thread_func(void *arg) {
    (void)arg;
    char cmd[64];

    while (running && list_running) {
        int len = snprintf(cmd, sizeof(cmd), "LIST %d\n", pid);
        if (len > 0 && server_fd >= 0) {
            (void)write(server_fd, cmd, (size_t)len);
        }

        // sleep 2 seconds but allow quick stop
        for (int i = 0; i < 20 && running && list_running; i++) {
            usleep(1000000);
        }
    }

    return NULL;
}

static int wait_result(int is_join) {
    // returns 1 success, 0 failure/timeout
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    ts.tv_sec += 5; // up to 5 seconds

    pthread_mutex_lock(&resp_mutex);
    last_result = 0;
    if (is_join) waiting_join = 1;
    else waiting_create = 1;
}

```

```

while (last_result == 0) {
    int rc = pthread_cond_timedwait(&resp_cond, &resp_mutex, &ts);
    if (rc == ETIMEDOUT) break;
}

int ok = (last_result == 1);

// If timed out, stop waiting flags.
if (is_join) waiting_join = 0;
else waiting_create = 0;

pthread_mutex_unlock(&resp_mutex);
return ok;
}

int main(void) {
    pid = getpid();
    install_client_handlers();

    // Reader thread will use this to interrupt blocking stdin reads when the game ends.
    main_thread_id = pthread_self();

    snprintf(client_fifo_path, sizeof(client_fifo_path), "client_%d_fifo", pid);

    // If previous run crashed and FIFO stayed
    unlink(client_fifo_path);

    if (mkfifo(client_fifo_path, 0666) < 0) {
        perror("mkfifo client");
        return 1;
    }

    server_fd = open(SERVER_FIFO, O_WRONLY | O_NONBLOCK);
    if (server_fd < 0) {
        perror("open server fifo");
        client_cleanup();
        return 1;
    }

    client_fd = open(client_fifo_path, O_RDONLY | O_NONBLOCK);
    if (client_fd < 0) {
        perror("open client fifo");
        client_cleanup();
        return 1;
    }

    pthread_t reader_thread;
    pthread_create(&reader_thread, NULL, read_thread_func, NULL);

    int choice;
    safe_print("1. Create game\n2. Join game\n> ");
    if (scanf("%d", &choice) != 1) {
        running = 0;
        pthread_join(reader_thread, NULL);
    }
}

```

```

        client_cleanup();
        return 1;
    }

pthread_t list_thread;
int list_thread_started = 0;

if (choice == 1) {
    int max;
    safe_print("Game name: ");
    scanf("%31s", game);
    safe_print("Max players (1-5): ");
    scanf("%d", &max);

    if (max < 1 || max > MAX_PLAYERS) {
        safe_print("Invalid number of players.\n");
        running = 0;
        pthread_join(reader_thread, NULL);
        client_cleanup();
        return 1;
    }

    char msg[128];
    snprintf(msg, sizeof(msg), "CREATE %s %d %d\n", game, max, pid);
    (void)write(server_fd, msg, strlen(msg));

    if (!wait_result(0)) {
        safe_print("Create failed (or timed out).\n");
        running = 0;
        pthread_join(reader_thread, NULL);
        client_cleanup();
        return 1;
    }

    exit_msg_len = (sig_atomic_t)snprintf(exit_msg, sizeof(exit_msg), "EXIT %s %d\n",
game, pid);
    have_game = 1;
}
else if (choice == 2) {
    safe_print("\nAvailable games list refreshes every 20 seconds.\n");

    list_running = 1;
    pthread_create(&list_thread, NULL, list_thread_func, NULL);
    list_thread_started = 1;

    safe_print("Enter game name to join: ");
    scanf("%31s", game);

    list_running = 0;
    pthread_join(list_thread, NULL);

    char msg[128];
    snprintf(msg, sizeof(msg), "JOIN %s %d\n", game, pid);
    (void)write(server_fd, msg, strlen(msg));
}

```

```

        if (!wait_result(1)) {
            safe_print("Join failed (or timed out).\n");
            running = 0;
            pthread_join(reader_thread, NULL);
            client_cleanup();
            return 1;
        }

        exit_msg_len = (sig_atomic_t)snprintf(exit_msg, sizeof(exit_msg), "EXIT %s %d\n",
game, pid);
        have_game = 1;
    }
    else {
        safe_print("Invalid choice\n");
        running = 0;
        pthread_join(reader_thread, NULL);
        client_cleanup();
        return 1;
    }

    // Clear stdin buffer after scanf
    int c; while ((c = getchar()) != '\n' && c != EOF) {}

    char word[32];
    while (running && !game_over) {
        safe_print("> ");
        if (!fgets(word, sizeof(word), stdin)) {
            // If we were interrupted because the game ended, just leave.
            if (game_over) break;
            if (errno == EINTR) continue;
            break;
        }

        word[strcspn(word, "\n")] = 0;

        if (game_over) break;

        if (strcmp(word, "EXIT") == 0) {
            char msg[128];
            snprintf(msg, sizeof(msg), "EXIT %s %d\n", game, pid);
            (void)write(server_fd, msg, strlen(msg));
            break;
        }

        char msg[128];
        snprintf(msg, sizeof(msg), "GUESS %s %d %s\n", game, pid, word);
        (void)write(server_fd, msg, strlen(msg));
    }

    running = 0;
    if (list_thread_started) {
        list_running = 0;
        pthread_join(list_thread, NULL);
    }

```

```
}  
pthread_join(reader_thread, NULL);  
  
client_cleanup();  
return 0;  
}
```

Протокол работы программы

Тестирование :

root@3e6923b56a50:/workspaces/Operating_Systems/bulls_cows_project/build# .

/client

1. Create game

2. Join game

> 1

Game name: g1

Max players (1-5): 2

Game created successfully

Player 1065 joined the game.

> www

Player 1006 guessed "www": Bulls=1, Cows=2

>

[Client] Game finished. Exiting input mode...

Player 1065 guessed "uwu": Bulls=3, Cows=0

Player 1065 WINS the game!

Game ended.

root@3e6923b56a50:/workspaces/Operating_Systems/bulls_cows_project

/build# ./client

1. Create game

2. Join game

> 2

Available games list refreshes every 20 seconds.

Active games:me to join:

Game: g1 Players: 1/2

> g1

Joined game successfully

Player 1065 joined the game.

Player 1006 guessed "www": Bulls=1, Cows=2

> uwu

>

[Client] Game finished. Exiting input mode...

Player 1065 guessed "uwu": Bulls=3, Cows=0

Player 1065 WINS the game!

Game ended.

Strace:

CEPBEP

```
Croot@3e6923b56a50:/workspaces/Operating_Systems/bulls_cows_project/build# strace
./server
execve("./server", ["/server"], 0x7ffce2ee86d0 /* 28 vars */) = 0
brk(NULL)                                = 0x56359d8ff000
arch_prctl(0x3001 /* ARCH_??? */, 0x7fff32c1c890) = -1 EINVAL (Invalid argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f9eec2c4000
access("/etc/ld.so.preload", R_OK)        = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=12460, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 12460, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f9eec2c0000
close(3)                                  = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0"..., 832) =
832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64)
= 784
pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0"..., 48,
848) = 48
pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\00{\f\225\\=\201\327\312\301P\32$\230\266\235"..., 68,
896) = 68
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0
```

```

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64)
= 784
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f9eec097000
mprotect(0x7f9eec0bf000, 2023424, PROT_NONE) = 0
mmap(0x7f9eec0bf000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x28000) = 0x7f9eec0bf000
mmap(0x7f9eec254000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1bd000) = 0x7f9eec254000
mmap(0x7f9eec2ad000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x215000) = 0x7f9eec2ad000
mmap(0x7f9eec2b3000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,
-1, 0) = 0x7f9eec2b3000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f9eec094000
arch_prctl(ARCH_SET_FS, 0x7f9eec094740) = 0
set_tid_address(0x7f9eec094a10) = 1766
set_robust_list(0x7f9eec094a20, 24) = 0
rseq(0x7f9eec0950e0, 0x20, 0, 0x53053053) = 0
mprotect(0x7f9eec2ad000, 16384, PROT_READ) = 0
mprotect(0x56359cdf8000, 4096, PROT_READ) = 0
mprotect(0x7f9eec2fe000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f9eec2c0000, 12460) = 0
rt_sigaction(SIGINT, {sa_handler=0x56359cdf48d7, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGTERM, {sa_handler=0x56359cdf48d7, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGHUP, {sa_handler=0x56359cdf48d7, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGQUIT, {sa_handler=0x56359cdf48d7, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGSEGV, {sa_handler=0x56359cdf489a, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGABRT, {sa_handler=0x56359cdf489a, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGFPE, {sa_handler=0x56359cdf489a, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGILL, {sa_handler=0x56359cdf489a, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGBUS, {sa_handler=0x56359cdf489a, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[],
sa_flags=SA_RESTORER|SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000,
sa_restorer=0x7f9eec0d9520}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
getpid() = 1766
unlink("server_fifo") = -1 ENOENT (No such file or directory)
mknodat(AT_FDCWD, "server_fifo", S_IFIFO|0666) = 0
openat(AT_FDCWD, "server_fifo", O_RDWR|O_NONBLOCK) = 3
rt_sigaction(SIGRT_1, {sa_handler=0x7f9eec128870, sa_mask=[],
sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x7f9eec0d9520},
NULL, 8) = 0
rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0

```

```

mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) =
0x7f9eeb893000
mprotect(0x7f9eeb894000, 8388608, PROT_READ|PROT_WRITE) = 0
getrandom("\x40\xff\x78\x8d\x16\x0c\xb0\x09", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x56359d8ff000
brk(0x56359d920000) = 0x56359d920000
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTID|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7f9eec093910, parent_tid=0x7f9eec093910, exit_signal=0, stack=0x7f9eeb893000, stack_size=0x7fff00, tls=0x7f9eec093640}, 88) = -1 ENOSYS (Function not implemented)
clone(child_stack=0x7f9eec092ef0, flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTID|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, parent_tid=[1767], tls=0x7f9eec093640, child_tidptr=0x7f9eec093910) = 1767
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
openat(AT_FDCWD, "server.log", O_WRONLY|O_CREAT|O_APPEND, 0666) = 4
lseek(4, 0, SEEK_END) = 735
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
newfstatat(4, "", {st_mode=S_IFREG|0644, st_size=735, ...}, AT_EMPTY_PATH) = 0
write(4, "[2025-12-20T19:43:49+00:00] Serv"..., 43) = 43
close(4) = 0
pselect6(4, [3], NULL, NULL, NULL, NULL) = 1 (in [3])
read(3, "CREATE g1 1 1978\n", 256) = 17
openat(AT_FDCWD, "words.txt", O_RDONLY) = 4
newfstatat(4, "", {st_mode=S_IFREG|0644, st_size=15, ...}, AT_EMPTY_PATH) = 0
read(4, "uwu\nara\nshinobi", 4096) = 15
read(4, "", 4096) = 0
close(4) = 0
openat(AT_FDCWD, "client_1978_fifo", O_WRONLY|O_NONBLOCK) = 4
write(4, "Game created successfully\n", 26) = 26
close(4) = 0
openat(AT_FDCWD, "server.log", O_WRONLY|O_CREAT|O_APPEND, 0666) = 4
lseek(4, 0, SEEK_END) = 778
newfstatat(4, "", {st_mode=S_IFREG|0644, st_size=778, ...}, AT_EMPTY_PATH) = 0
write(4, "[2025-12-20T19:44:23+00:00] Game"..., 41) = 41
close(4) = 0
pselect6(4, [3], NULL, NULL, NULL, NULL) = 1 (in [3])
read(3, "GUESS g1 1978 uwu\n", 256) = 18
openat(AT_FDCWD, "client_1978_fifo", O_WRONLY|O_NONBLOCK) = 4
write(4, "Wrong word length\n", 18) = 18
close(4) = 0
pselect6(4, [3], NULL, NULL, NULL, NULL) = 1 (in [3])
read(3, "GUESS g1 1978 shinobi\n", 256) = 22
openat(AT_FDCWD, "client_1978_fifo", O_WRONLY|O_NONBLOCK) = 4
write(4, "Player 1978 guessed \"shinobi\": B"..., 47) = 47
close(4) = 0
openat(AT_FDCWD, "client_1978_fifo", O_WRONLY|O_NONBLOCK) = 4
write(4, "Player 1978 WINS the game!\n", 27) = 27
close(4) = 0
openat(AT_FDCWD, "client_1978_fifo", O_WRONLY|O_NONBLOCK) = 4
write(4, "Game ended.\n", 12) = 12
close(4) = 0
openat(AT_FDCWD, "server.log", O_WRONLY|O_CREAT|O_APPEND, 0666) = 4
lseek(4, 0, SEEK_END) = 819
newfstatat(4, "", {st_mode=S_IFREG|0644, st_size=819, ...}, AT_EMPTY_PATH) = 0
write(4, "[2025-12-20T19:44:32+00:00] Game"..., 37) = 37
close(4) = 0
pselect6(4, [3], NULL, NULL, NULL, NULL^C) = ? ERESTARTNOHAND (To be restarted if no handler)
strace: Process 1766 detached

```

CLIENT

```
root@3e6923b56a50:/workspaces/Operating_Systems/bulls_cows_project
/build# strace ./client
execve("./client", ["/client"], 0x7ffe17e96a50 /* 28 vars */) = 0
brk(NULL) = 0x5628f32ab000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffde4f4b0) = -1 EINVAL (Invalid argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f373dbd3000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=12460, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 12460, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f373dbcf000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0"..., 832) =
832
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 784, 64)
= 784
pread64(3, "\4\0\0\0\0\0\0\0\5\0\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0\0"..., 48,
848) = 48
pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\00{\f\225\\=\201\327\312\301P\32$\230\266\235"..., 68,
896) = 68
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 784, 64)
= 784
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f373d9a6000
mprotect(0x7f373d9ce000, 2023424, PROT_NONE) = 0
mmap(0x7f373d9ce000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x28000) = 0x7f373d9ce000
mmap(0x7f373db63000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1bd000) = 0x7f373db63000
mmap(0x7f373dbbc000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x215000) = 0x7f373dbbc000
mmap(0x7f373dbc2000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,
-1, 0) = 0x7f373dbc2000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f373d9a3000
arch_prctl(ARCH_SET_FS, 0x7f373d9a3740) = 0
set_tid_address(0x7f373d9a3a10) = 1978
set_robust_list(0x7f373d9a3a20, 24) = 0
rseq(0x7f373d9a40e0, 0x20, 0, 0x53053053) = 0
mprotect(0x7f373dbbc000, 16384, PROT_READ) = 0
mprotect(0x5628f21ce000, 4096, PROT_READ) = 0
mprotect(0x7f373dc0d000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f373dbcf000, 12460) = 0
getpid() = 1978
rt_sigaction(SIGINT, {sa_handler=0x5628f21cb686, sa_mask=[INT],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGTERM, {sa_handler=0x5628f21cb686, sa_mask=[TERM],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGHUP, {sa_handler=0x5628f21cb686, sa_mask=[HUP],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGQUIT, {sa_handler=0x5628f21cb686, sa_mask=[QUIT],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
```

```

rt_sigaction(SIGSEGV, {sa_handler=0x5628f21cb686, sa_mask=[SEGV],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGABRT, {sa_handler=0x5628f21cb686, sa_mask=[ABRT],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGFPE, {sa_handler=0x5628f21cb686, sa_mask=[FPE],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGILL, {sa_handler=0x5628f21cb686, sa_mask=[ILL],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGBUS, {sa_handler=0x5628f21cb686, sa_mask=[BUS],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[PIPE],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f373d9e8520}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGUSR1, {sa_handler=0x5628f21cb6f9, sa_mask=[], sa_flags=SA_RESTORER,
sa_restorer=0x7f373d9e8520}, NULL, 8) = 0
unlink("client_1978_fifo") = -1 ENOENT (No such file or directory)
mknodat(AT_FDCWD, "client_1978_fifo", S_IFIFO|0666) = 0
openat(AT_FDCWD, "server_fifo", O_WRONLY|O_NONBLOCK) = 3
openat(AT_FDCWD, "client_1978_fifo", O_RDONLY|O_NONBLOCK) = 4
rt_sigaction(SIGRT_1, {sa_handler=0x7f373da37870, sa_mask=[],
sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x7f373d9e8520},
NULL, 8) = 0
rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) =
0x7f373d1a2000
mprotect(0x7f373d1a3000, 8388608, PROT_READ|PROT_WRITE) = 0
getrandom("\xfa\xba\x48\xcd\x92\x51\xba\xfa", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x5628f32ab000
brk(0x5628f32cc000) = 0x5628f32cc000
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CL
ONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7f373d9a2910,
parent_tid=0x7f373d9a2910, exit_signal=0, stack=0x7f373d1a2000, stack_size=0x7fff00,
tls=0x7f373d9a2640}, 88) = -1 ENOSYS (Function not implemented)
clone(child_stack=0x7f373d9a1ef0,
flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETT
LS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, parent_tid=[1979], tls=0x7f373d9a2640,
child_tidptr=0x7f373d9a2910) = 1979
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
newfstatat(1, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x1), ...},
AT_EMPTY_PATH) = 0
write(1, "1. Create game\n", 151. Create game
) = 15
write(1, "2. Join game\n", 132. Join game
) = 13
write(1, "> ", 2> ) = 2
newfstatat(0, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x1), ...},
AT_EMPTY_PATH) = 0
read(0, 1
"1\n", 1024) = 2
write(1, "Game name: ", 11Game name: ) = 11
read(0, g1
"g1\n", 1024) = 3
write(1, "Max players (1-5): ", 19Max players (1-5): ) = 19
read(0, 1
"1\n", 1024) = 2
write(3, "CREATE g1 1 1978\n", 17) = 17

```

```

futex(0x5628f21cf188, FUTEX_WAIT_BITSET_PRIVATE|FUTEX_CLOCK_REALTIME, 0,
{tv_sec=1766259868, tv_nsec=779420441}, FUTEX_BITSET_MATCH_Game created successfully

> ) = 0
futex(0x5628f21cf120, FUTEX_WAKE_PRIVATE, 1) = 0
write(1, "> ", 2) = 2
read(0, uwu
"uwu\n", 1024) = 4
write(3, "GUESS g1 1978 uwu\n", 18) = 18
write(1, "> ", 2) = 2
Wrong word length

> shinobi
"shinobi\n", 1024) = 8
write(3, "GUESS g1 1978 shinobi\n", 22) = 22
write(1, "> ", 2) = 2
read(0,
[Client] Game finished. Exiting input mode...
Player 1978 guessed "shinobi": Bulls=7, Cows=0
Player 1978 WINS the game!
Game ended.

> 0x5628f32ab7d0, 1024) = ? ERESTARTSYS (To be restarted if SA_RESTART is
set)
--- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_TKILL, si_pid=1978, si_uid=0} ---
rt_sigreturn({mask=[]}) = -1 EINTR (Interrupted system call)
futex(0x7f373d9a2910, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 1979, NULL,
FUTEX_BITSET_MATCH_ANY) = 0
close(4) = 0
close(3) = 0
unlink("client_1978_fifo") = 0
unlink("client_1978_fifo") = -1 ENOENT (No such file or directory)
exit_group(0) = ?
+++ exited with 0 +++

```

Вывод

В ходе работы была разработана и протестирована клиент-серверная консольная игра «Быки и коровы», реализованная средствами межпроцессного взаимодействия POSIX. Для обмена данными между процессами использованы именованные каналы FIFO: общий канал `server_fifo` для передачи команд серверу и индивидуальные каналы `client_<pid>_fifo` для доставки ответов каждому клиенту. Сервер поддерживает создание игр, подключение игроков, получение списка доступных игр, обработку попыток угадывания и выход из игры, а также ведёт журнал событий в файл `server.log`.

Реализация обеспечивает корректное разделение ответственности: сервер централизованно хранит состояние игр и управляет очередностью ходов, клиент отвечает за пользовательский интерфейс и отправку команд. Для повышения отзывчивости клиента используется отдельный поток чтения, что позволяет получать сообщения от сервера независимо от ввода пользователя. Также предусмотрена корректная обработка завершения работы (закрытие дескрипторов и удаление FIFO), что уменьшает вероятность накопления “мусорных” каналов в файловой системе.

Таким образом, поставленная задача реализована: получена работоспособная программа с использованием POSIX IPC, демонстрирующая принципы клиент-серверного взаимодействия, синхронизации и обработки событий. Проект может быть использован для демонстрации практических навыков работы с FIFO, системными вызовами ввода-вывода и организации обмена сообщениями между процессами.